

Cấu trúc dữ liệu

Thực hành 6 - Danh sách liên kết (phần 2)

Bài tập 1: Xác định các mệnh đề sau là đúng hay sai

1. Mỗi nút trong DSLK có hai thành phần: một là để chứa dữ liệu, hai là để chứa địa chỉ. → **True**
2. Trong DSLK, thứ tự của các nút được quyết định bởi thứ tự của chúng khi được thêm vào DSLK. → **False**
3. Trong DSLK, địa chỉ của các nút là tuần tự. → **False**
4. Trong DSLK, con trỏ của nút cuối cùng trỏ về chính nó. → **False**
5. Giả sử nút được khởi tạo ở dạng `info-next`, và con trỏ `current` chỉ vào một nút trong DSLK, để con trỏ `current` chỉ vào nút tiếp theo của DSLK ta sử dụng cú pháp

```
current = current.next
```

→ **False**

6. Các bước chèn nút vào DSLK là: khởi tạo nút, cập nhật giá trị biến của nút, cập nhật giá trị con trỏ của nút. → **False**
7. Danh sách liên kết đơn có thể được duyệt từ hai hướng. → **False**
8. Trong DSLK, nút luôn được chèn vào vị trí đầu tiên hoặc cuối cùng. → **False**
9. Không nên dùng con trỏ `head/first` để duyệt DSLK. → **True**

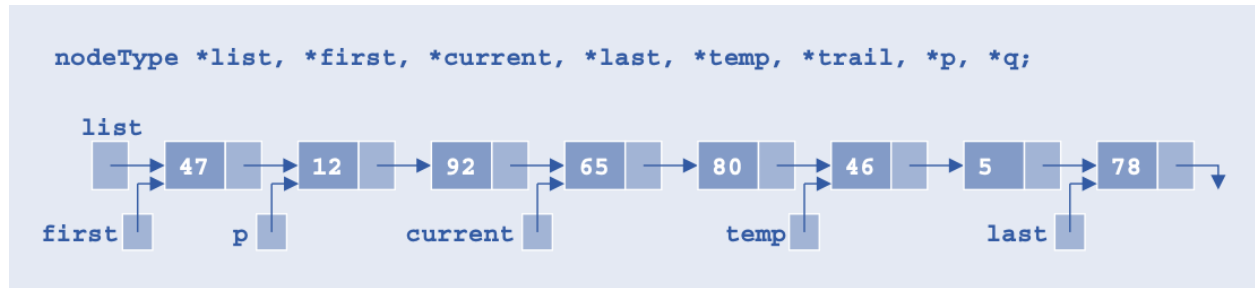
Bài tập 2: Giải thích ý nghĩa của các câu lệnh sau

1. `cout << p->info;`
`// in ra giá trị của biến info tại địa chỉ p.`
2. `q = p->link;`
`// gán giá trị của con trỏ link tại địa chỉ p cho con trỏ q`
`cout << q->info << " " << current->info;`
`// in ra giá trị của biến info tại vị trí q và current`
3. `cout << current->link->info;`
`// in ra giá trị của biến info tại nút tiếp theo nút có địa chỉ là current.`
4. `trail = current->link->link;`
`// gán giá trị của con trỏ của nút tiếp theo cho con trỏ trail`
`trail->link = nullptr;`
`// con trỏ tại nút tiếp theo chỉ về nullptr`
`// nút tiếp theo là nút cuối cùng của DSLK`
`cout << trail->info;`
`// in ra giá trị của biến info tại nút cuối cùng`
5. `cout << last->link->info;`
`// in ra giá trị của biến info của nút tiếp theo`
6. `q = current->link;`
`// gán giá trị của con trỏ của nút hiện tại cho con trỏ q`
`cout << q->link->link->info;`
`// in ra giá trị của biến info của nút cách nút hiện tại 2 nút.`

Bài tập 3: Giải thích ý nghĩa của các câu lệnh sau, biết **p** là con trỏ và nút có cấu trúc **info-link**.

1. `p->link->link == current`
// so sánh giá trị của con trỏ `current` với con trỏ tại nút tiếp theo tính từ địa chỉ `p`
// so sánh giá trị của con trỏ ngay sau `p` với con trỏ `current`
2. `first->link->link->info == 92`
// so sánh giá trị của biến `info` của nút thứ 3 trong DSLK với 92
3. `temp->link == 0`
// so sánh giá trị của con trỏ tại địa chỉ `temp` với 0
4. `last->link == nullptr`
// so sánh giá trị của con trỏ tại vị trí `last` với `nullptr`
5. `list->link == p`
// so sánh giá trị của con trỏ tại vị trí `list` với con trỏ `p`
6. `p->link->link->link->info == temp->info`
// so sánh giá trị của biến `info` của nút cách vị trí `p` 2 nút với giá trị của biến `info` tại con trỏ `temp`
// so sánh giá trị của biến `info` của nút thứ 3 sau `p` với giá trị của biến `info` tại con trỏ `temp`

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10
p			?						



Bài tập 4: Viết câu lệnh C++ để thực thi các yêu cầu sau:

1. Đổi giá trị **info** của nút thứ 3 thành 24.

```
p->link->info = 24;
```

2. Con trỏ **q** chỉ tới nút có info bằng 80.

```
q = current->link;
```

3. Con trỏ **trail** chỉ tới nút ở phía trước con trỏ **current**.

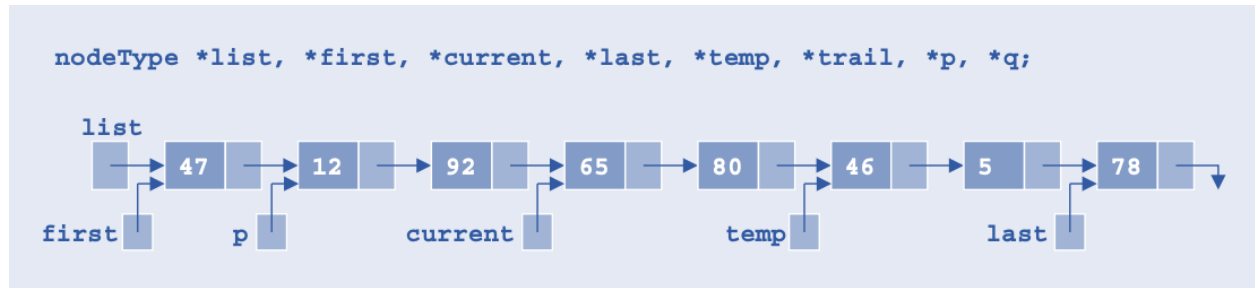
```
trial = p->link;
```

4. Đổi giá trị **info** của nút kết cuối DSLK thành 54.

```
temp->link->info = 54;
```

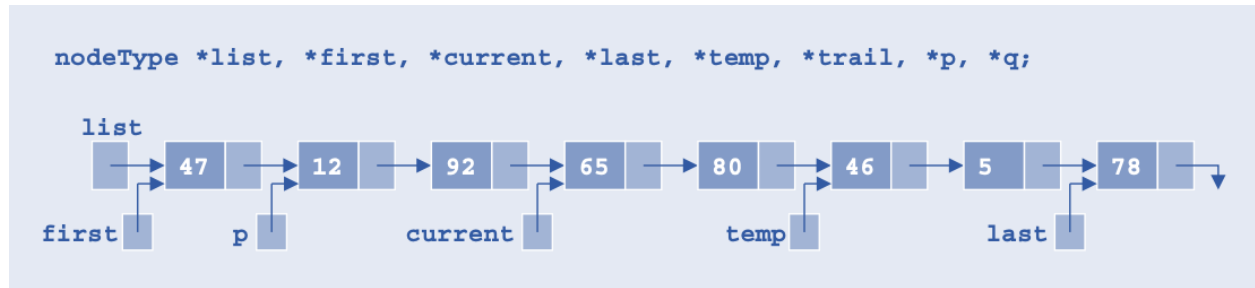
5. Dùng con trỏ **first** để duyệt toàn bộ DSLK.

```
while (first != nullptr)
{
    first = first ->link;
}
```



Bài tập 5: Xác định tính hợp lệ của các câu lệnh sau

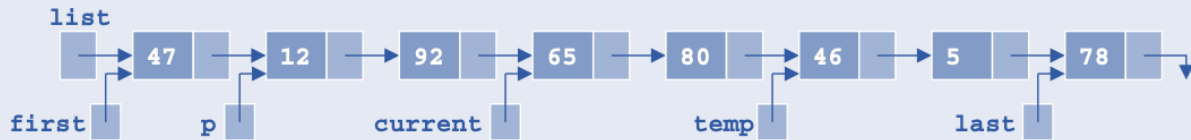
1. `p = list->link;` → **Valid**
2. `first = list;` → **Valid**
3. `temp->link = nullptr;` → **Valid**
4. `current->link = temp->info;` → **Invalid**
5. `p = *last;` → **Invalid**
`p` là con trỏ, `*last` là struct
6. `first = 90;` → **Invalid**
`first` là con trỏ, chỉ chứa địa chỉ, không chứa số int.
7. `p->link->info = current->info;` → **Valid**
8. `current->info = temp->link;` → **Invalid**
9. `*list = *temp;` → **Valid**
10. `temp->link = last->link->link;` → **Invalid**
 Báo lỗi execution error, `last->link = nullptr` nên `last->link->link` không tồn tại.
11. `cout << trail->link->link->info;` → **Invalid**
 Báo lỗi execution error, con trỏ `trail` chỉ mới được khởi tạo, chưa có giá trị nên `trail->link->link->info` không tồn tại.



Bài tập 6: Xác định tính hợp lệ của đoạn code sau. Nếu hợp lệ thì output của đoạn code này là gì, nếu không hợp lệ thì giải thích lý do.

```
1. // gán địa chỉ của con trỏ current cho q; q --> 65
   q = current;
   // con trỏ current chỉ tới nút tiếp theo, current --> 80
   current = current->link;
   // gán giá trị của con trỏ link tại địa chỉ temp cho con
   // trỏ link tại địa chỉ current; current->link --> 5; nút 46
   // bị treo
   current->link = temp->link;
   // gán giá trị/ khởi tạo con trỏ trail; trail --> 5
   trail = current->link;
   // gán giá trị của con trỏ link tại địa chỉ trail cho con
   // trỏ current; current --> 78
   current = trail->link;
   // 65 5 78
   cout << q->info << " " << trail->info
        << " " << current->info << endl;
```

```
nodeType *list, *first, *current, *last, *temp, *trail, *p, *q;
```



```
2. // gán giá trị của con trỏ link tại địa chỉ temp cho con
    // trỏ trail; trail --> 5
    trail = temp->link;
    // con trỏ temp trỏ tới nút thứ 3 sau vị trí hiện tại
    // temp --> 78
    temp = temp->link->link;
    // gán giá trị của con trỏ temp cho current
    current = temp;
    // con trỏ current chỉ tới nút tiếp theo; current = nullptr
    // vì đang trỏ tới nút cuối cùng
    current = current->link;
    // con trỏ link tại địa chỉ trail chỉ tới nullptr
    // nút 5 sẽ là nút cuối cùng của DSLK
    trail->link = last->link;
    // 78 78
    cout << current->info << " " << last->info << endl;
```